Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

Aula 07 — Comunicação entre Componentes

Comunicação Direta e Indireta — props, callbacks e prop drilling

Prof.: MSc. Alex Junior Nunes
pfsalexnunes@gmail.com

CONCEITO

Por Que os Componentes Precisam se Falar?

Até agora, cada tela era praticamente um componente só. Conforme o app cresce, ele vira uma ÁRVORE de componentes pequenos — e eles precisam trocar informação entre si.

D

Direta

Entre um componente e seu PAI ou FILHO imediato — a relação mais simples.

I

Indireta

Entre componentes sem relação direta (irmãos, ou vários níveis de distância) — precisa de um intermediário.

!

O problema real

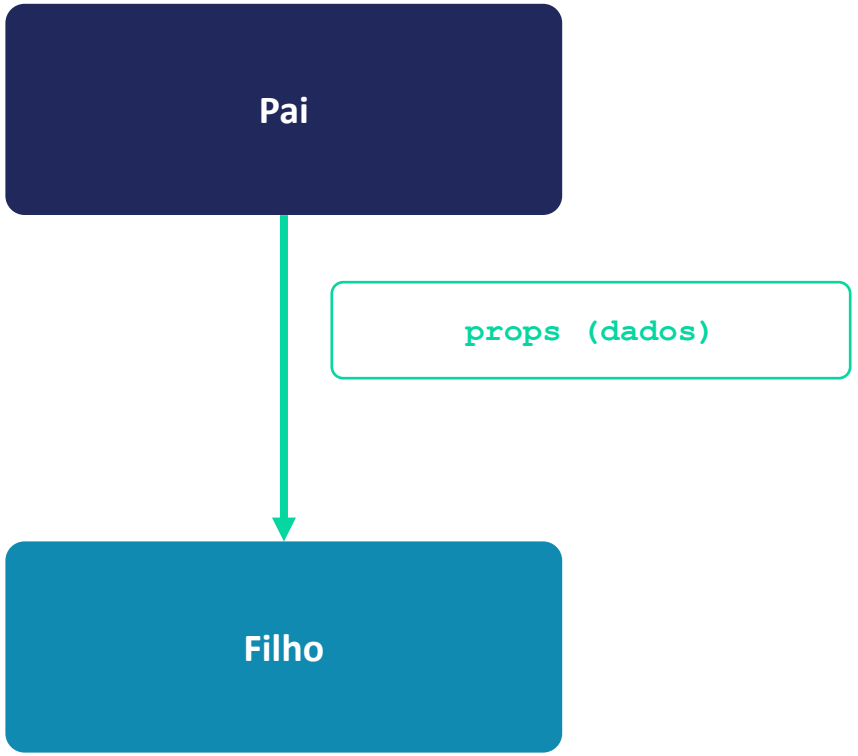
Quanto mais distante, mais componentes no meio do caminho só "repassam" informação sem usá-la.

Convenção usada nesta aula: "direta" = existe uma ligação direta na árvore (pai ↔ filho, em qualquer sentido); "indireta" = precisa de um intermediário (prop drilling, irmãos). É um critério didático — outros materiais usam o sentido do fluxo de dados e chamam o callback filho→pai de indireto.

Direta: Pai → Filho (props)

```
<PerfilCard
  nome="Ana"
  cargo="UX Designer"
/>

function PerfilCard(
  { nome, cargo }
) { ... }
```



Via de MÃO ÚNICA: o pai manda o dado, o filho só lê. O filho nunca altera a prop que recebeu.

Direta: Filho → Pai (callback)

```
// no pai

function aoCurtir(nome) {

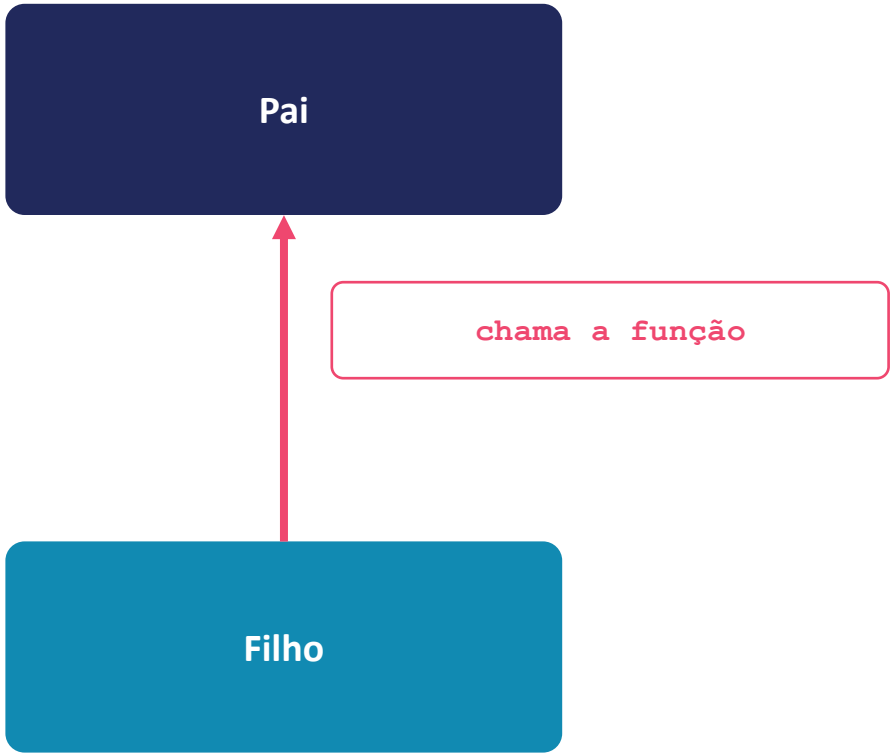
  Alert.alert('Curtida!',
nome);

}


<BotaoCurtir

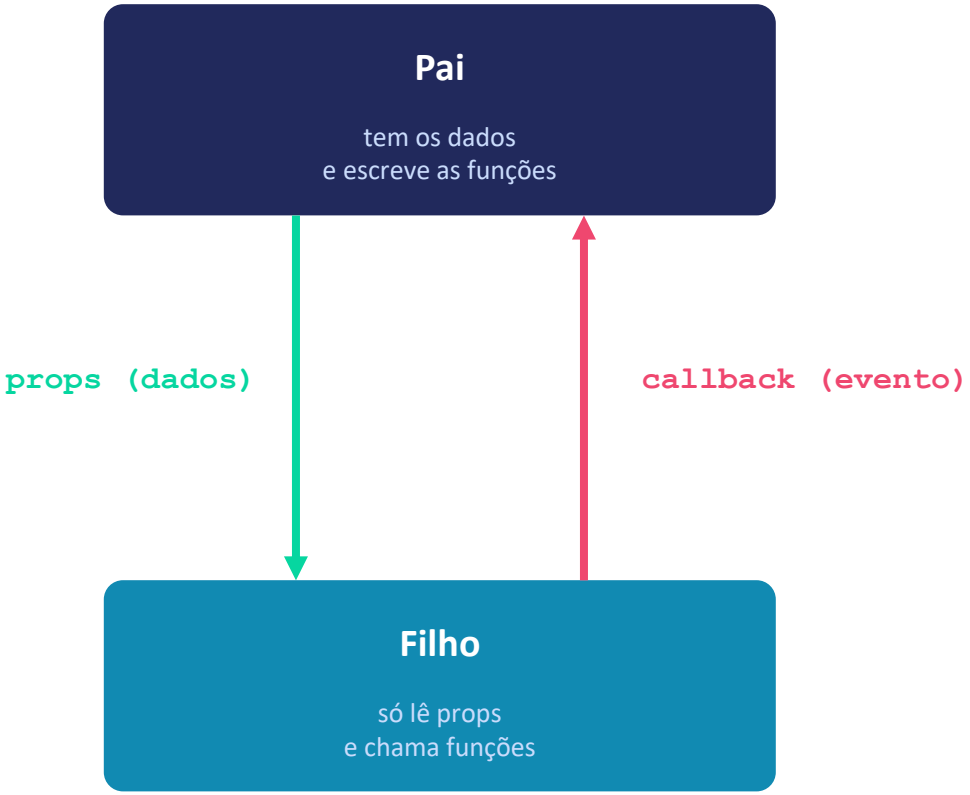
  aoPressionar={aoCurtir}

/>
```



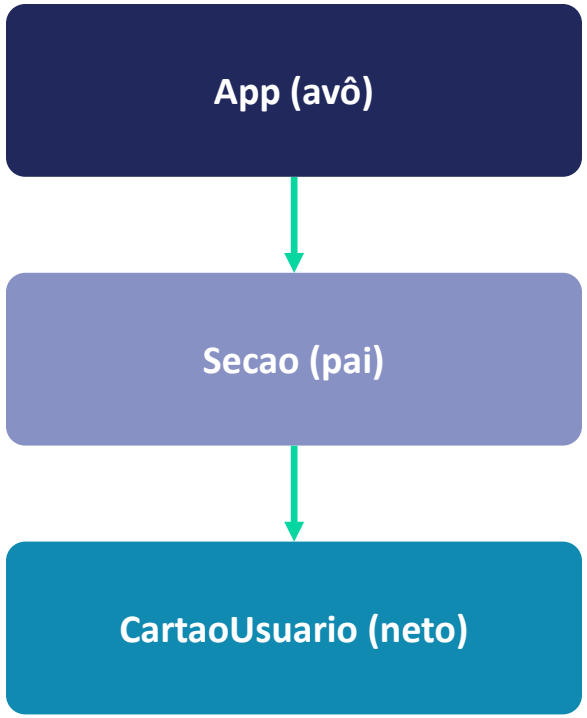
O pai escreve a função e passa como prop. O filho só CHAMA — nunca sabe (nem precisa saber) o que ela faz por dentro.

Combinando os Dois: Dados Descem, Eventos Sobem



É o mesmo padrão de qualquer lista, feed ou carrinho de compras real — um item que recebe dados e avisa quando algo acontece.

Indireta: Prop Drilling



`usuario, aoTocarPerfil`

"Secao" não usa `usuario` para nada — só repassa adiante.

`usuario, aoTocarPerfil`

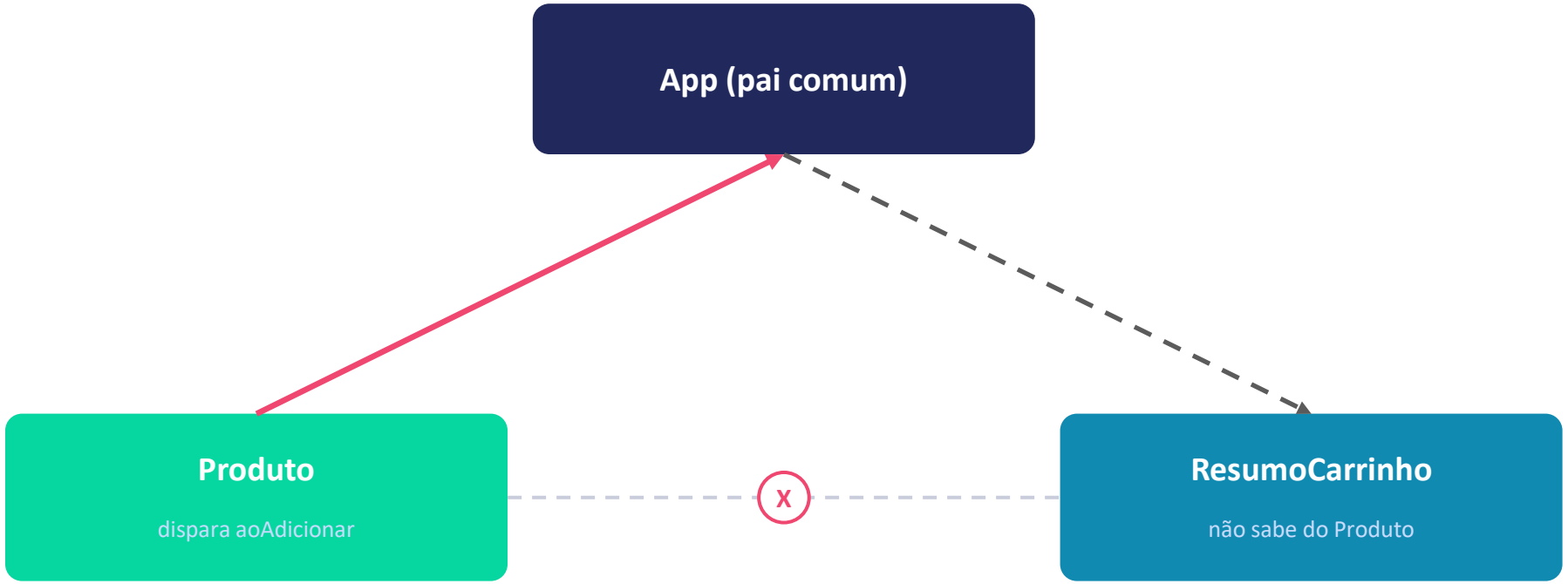
O problema:

quanto mais níveis entre quem TEM o dado e quem PRECISA dele, mais componentes "inocentes" no meio precisam repassar props que nem usam.

Isso deixa o código mais frágil — mudar um dado lá em cima obriga a mexer em todo o caminho até embaixo.

Isso se chama PROP DRILLING — e é exatamente o que o Redux (Aula 11) existe para resolver.

Indireta: Irmãos via Ancestral Comum



Sem Hooks/Redux ainda, o App só confirma com um Alert que recebeu o evento — o ResumoCarrinho não atualiza sozinho por enquanto.

Por Que Isso Vai Doer (um pouco)

Prop drilling funciona bem com 2-3 níveis. Mas em um app real, com 5, 6, 7 níveis de componentes, repassar prop por prop vira um trabalho manual, repetitivo e fácil de errar.

- Toda vez que um componente do meio muda, você precisa lembrar de atualizar as props que ele repassa
- Componentes que não usam o dado ficam "sujos" de props que só existem para passar adiante
- Na Aula 11 (Redux), os componentes vão poder ler e escrever num estado global, sem precisar de toda essa cadeia de props

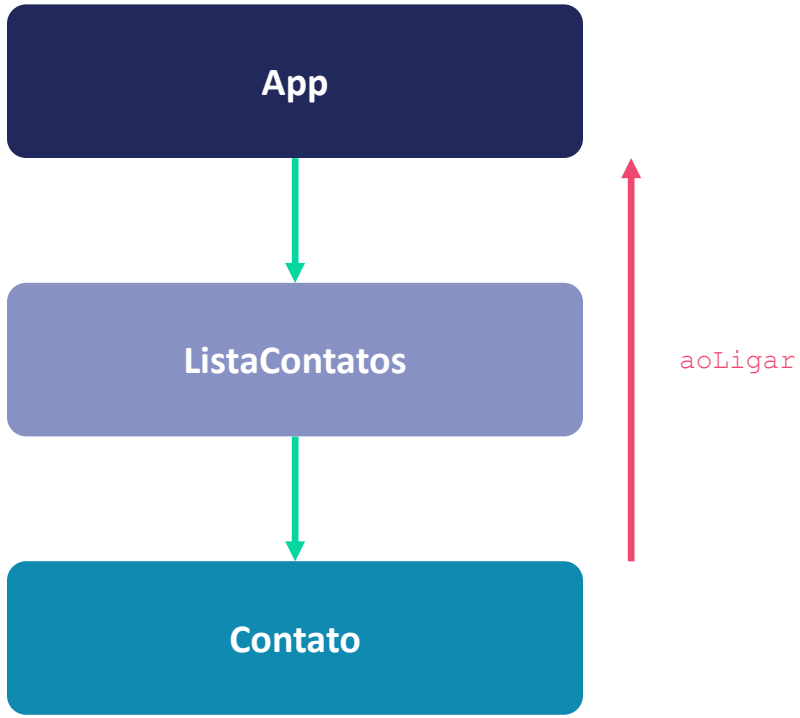
Por hoje, o objetivo é sentir a dor do prop drilling — a solução vem depois.

DESAFIO

Desafio da Aula: Árvore de Componentes

Em duplas, montem App → ListaContatos → Contato, praticando as duas comunicações (sem useState):

- [ ] App: array CONTATOS + função aoLigar(contato) com Alert
- [ ] ListaContatos (pai): só repassa contatos e aoLigar
- [ ] Contato (neto): mostra nome/telefone + botão que chama aoLigar



`Arquivo: 06-desafio-arvore-componentes.js · Gabarito: 06-desafio-arvore-componentes-solucao.js`

O Que Levamos de Hoje

- Comunicação direta: props descem (pai → filho), callbacks sobem (filho → pai)
- O filho nunca altera a prop recebida — só o pai decide o que fazer
- Comunicação indireta: prop drilling repassa dados por vários níveis
- Irmãos só "se falam" através de um ancestral comum
- Prop drilling funciona, mas dói em apps grandes — é o que motiva o Redux (Aula 11)

Próxima aula: prática — melhorando um aplicativo básico já estilizado.